PROJECT TITLE

ENFORCING MANDATORY FIELDS USING UI POLICIES AND MIGRATING CHANGES WITH UPDATE SETS

SUBMITTED BY

DEBORA SHALINI .T (920123104006)

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

BHARATH NIKETAN ENGINEERING COLLEGE , AUNDIPATTI

ANNA UNIVERSITY ,CHENNAI 600 025

APRIL/MAY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report on **"ENFORCING MANDATORY FIELDS USING UI POLICIES AND MIGRATING CHANGES WITH UPDATE SETS"** is the Bonafide work of **DEBORA SHALINI.T (920123104006)** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Project Supervisor	**Head of the Department**
Mrs.V.Victoriya.ME.,	**Mrs.R.Mahalakshmi.M.E.,**
Department of Computer Science	**Department of Computer Science**
and Engineering,	**and Engineering,**
Bharath Niketan Engineering College	**Bharath Niketan Engineering College**
Aundipatti-625 536.	**Aundipatti-625 536.**

Submitted for the project Viva-Voice examination held on -

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We thank the most graceful creator of the universe, our almighty GOD, who ideally supported us thought this project.

At this moment of having successfully completed our project, we wish to convey our sincere thanks to the management and our Chairman Dr. S. MOHAN who provide all the facilities to us.

We would like to express our sincere thanks to our Principal Dr.P.V.ARUL KUMAR, M.E, M.B.A, Ph.D., for letting us to do our project and offering adequate duration in completing our project.

We are also grateful to Mrs.R.MAHALAKSHMI,M.E., our Head of the Department for his constructive suggestions during our project with deep sense of gratitude.

We extend our sincere thanks to our guide Mrs.V.VICTORIYA, M.E., Assistant Professor, Department of Computer Science and Engineering, who is our light house in the vast ocean of learning with her inspiring guideless and encouragement to complete the project.

We would like to express our gratitude to all the teaching and nonteaching staff members of Computer Science and Engineering Department and our friends for that kind help extended to us.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 PROJECT OVERVIEW

1.2 PURPOSE

2. IDEATION PHASE

2.1 PROBLEM STATEMENT

2.2 EMPATHY MAP CANVAS

2.3 BRAINSTORMING

3. REQUIREMENT ANALYSIS

3.1 CUSTOMER JOURNEY MAP

3.2 SOLUTION REQUIREMENT

3.3 DATA FLOW DIAGRAM

3.4 TECHNOLOGY STACK

4. PROJECT DESIGN

4.1 PROBLEM SOLUTION FIT

4.2 PROPOSED SOLUTION

4.3 SOLUTION ARCHITECTURE

5. PROJECT PLANNING & SCHEDULING

5.1 PROJECT PLANNING

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 PERFORMANCE TESTING

6.2 USER ACCEPTANCE TESTING

7. RESULTS

7.1 OUTPUT SCREENSHOTS

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

11.1 GITHUB & PROJECT DEMO LINK

Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

---

1.INTRODUCTION

1.1 Project Overview

In modern enterprise applications, maintaining accurate and consistent data is essential for efficient operations and decision-making. This project focuses on enforcing mandatory fields in forms using UI Policies to ensure that users provide all required information before submission.

Additionally, the project addresses the challenge of migrating configurations and customizations across different environments (development, testing, and production) using Update Sets. This ensures a structured and error-free deployment process.

The combination of UI Policies and Update Sets provides a robust solution for improving data quality and maintaining system consistency.

1.2 Purpose

The main purpose of this project is to:

- Ensure all required fields are filled before form submission
- Improve data accuracy and reliability
- Reduce manual errors during data entry
- Simplify the migration of system changes
- Maintain consistency across multiple environments

2. IDEATION PHASE

2.1 Problem Statement

Organizations often face issues with incomplete or incorrect data due to users skipping important fields. This leads to:

- Poor data quality
- Errors in reports and workflows
- Increased manual correction efforts

Additionally, migrating changes manually between environments can:

- Cause inconsistencies
- Lead to deployment errors
- Increase development time

2.2 Empathy Map Canvas

Users Think: The system should be simple and guide them clearly
Users Feel: Frustrated when errors occur after submission
Users Say: "I want a system that prevents mistakes while filling forms"

Users Do: Enter data quickly, sometimes skipping fields

This highlights the need for real-time validation and user-friendly enforcement mechanisms

2.3 Brainstorming

Different approaches were considered:

- Client-side scripting
- Server-side validation
- UI Policies
- Data Policies

After evaluation, UI Policies were chosen because:

- They provide dynamic control on the user interface
- They are easy to configure without heavy coding
- They improve user experience

Update Sets were selected for migration because:

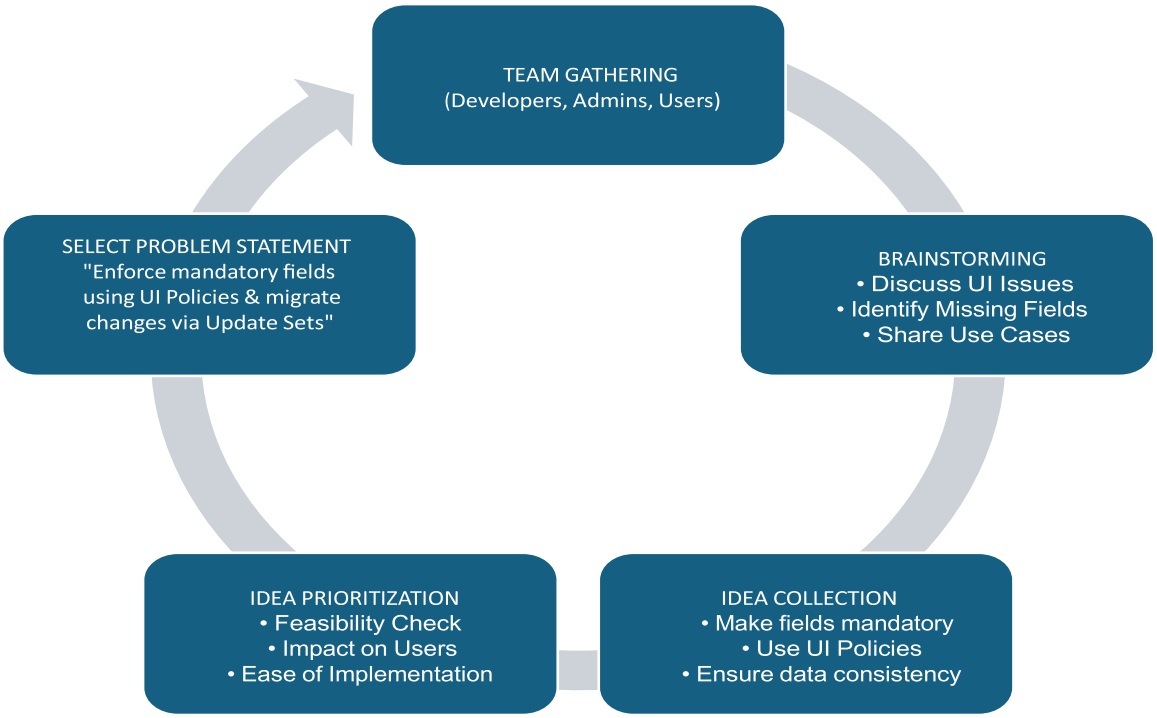
- They track configuration changes
- They allow smooth transfer between instances
- They reduce deployment risks

Ideation Phase

Brainstorm & Idea Prioritization

Brainstorm & Idea Prioritization is a two-part process used to generate ideas and then decide which ones are most valuable or worth pursuing.

Step-1: Team Gathering, Collaboration and Select the Problem Statement



Step-2: Brainstorm, Idea Listing and Grouping

BRAINSTORMING

- Identify form issues
- Discuss data gaps
- Share real-time problems

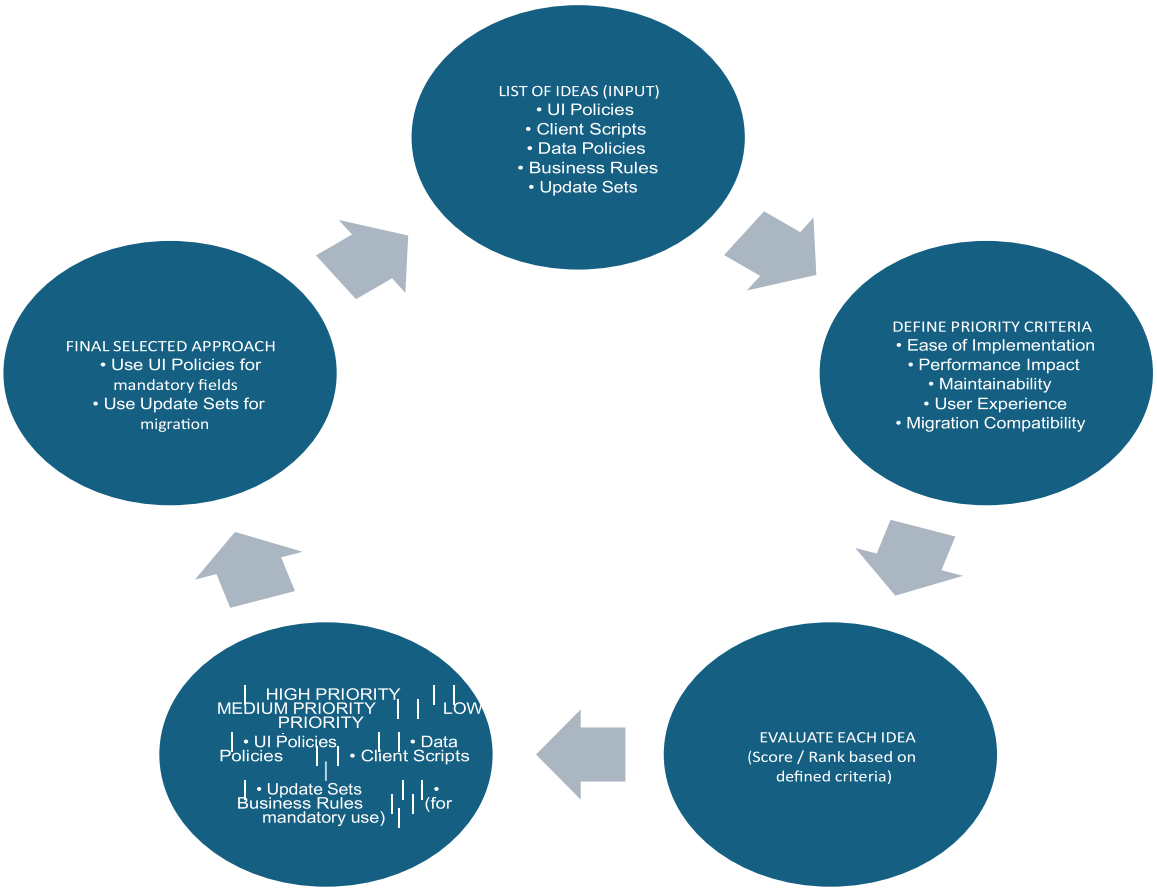
IDEA LISTING

- Make fields mandatory
- Use UI Policies
- Use Client Scripts
- Apply Data Policies
- Track changes via Update

IDEA GROUPING

- UI CONTROL GROUP
- VALIDATION GROUP
- MIGRATION GROUP

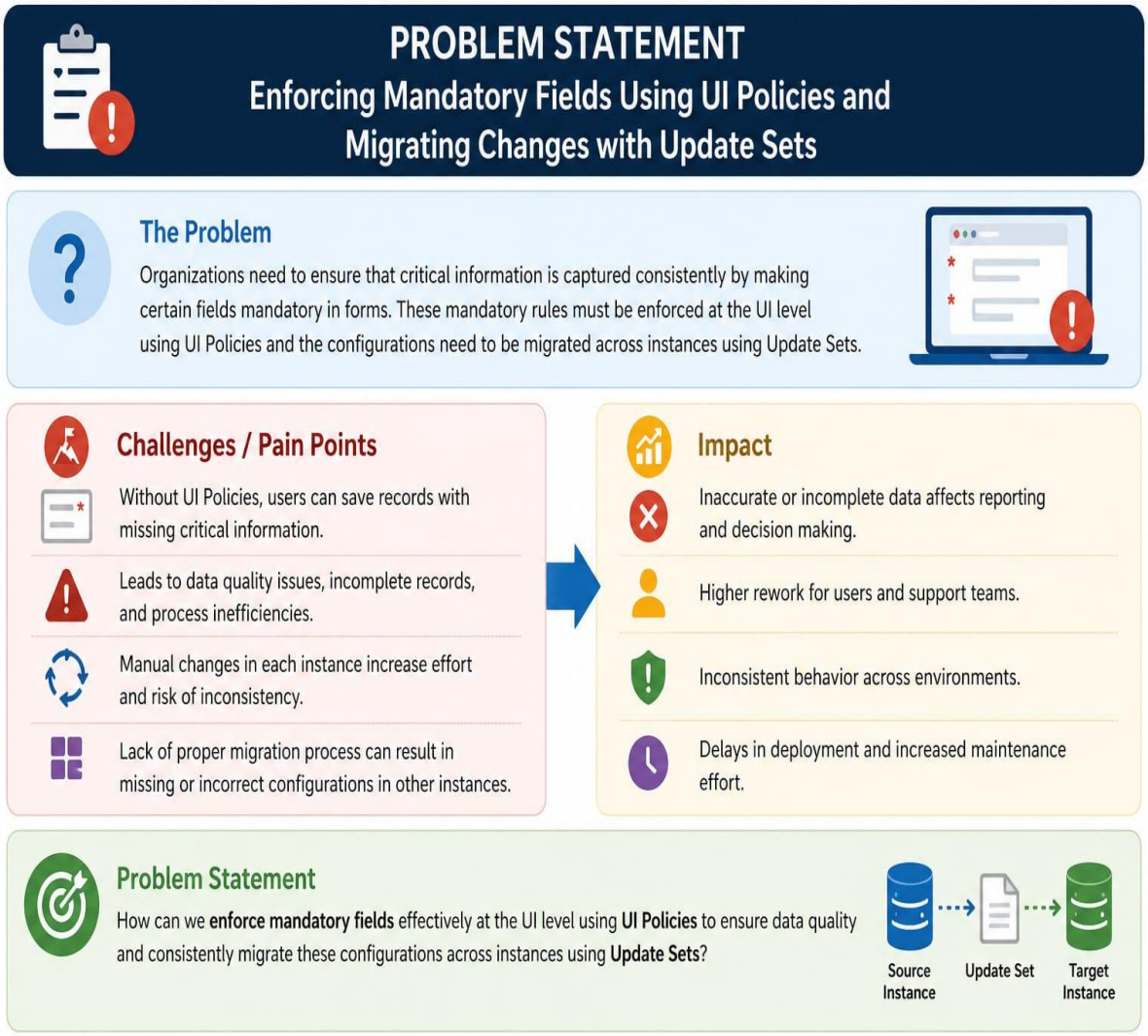
Step-3: Idea Prioritization



Define the Problem Statements

Customer Problem Statement

Customer Problem Support refers to the process of helping customers identify, address, and resolve issues they face while using a product or service, ensuring a positive experience and satisfaction.



Empathize & Discover

EMPATHY MAP

An Empathy Map is a visual tool used to understand a customer or user by capturing what they think, feel, say, and do, along with their needs and pain points. It helps teams better understand user experiences and design solutions that truly meet their needs.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. User logs into the system
2. Opens a form
3. Starts entering data
4. UI Policies enforce mandatory fields dynamically
5. User fills all required fields
6. Form is submitted successfully
7. Data is stored in the system

3.2 Solution Requirement

The system must:

- Dynamically enforce mandatory fields
- Provide visual indicators for required fields
- Prevent submission if mandatory fields are empty
- Support easy configuration of UI Policies
- Enable migration using Update Sets
- Maintain system performance

3.3 Data Flow Diagram

Flow:

User Input → UI Policy Validation → Application Processing → Database Storage → Output Display

- UI Policies validate inputs at the front-end
- Only valid data is passed to the backend

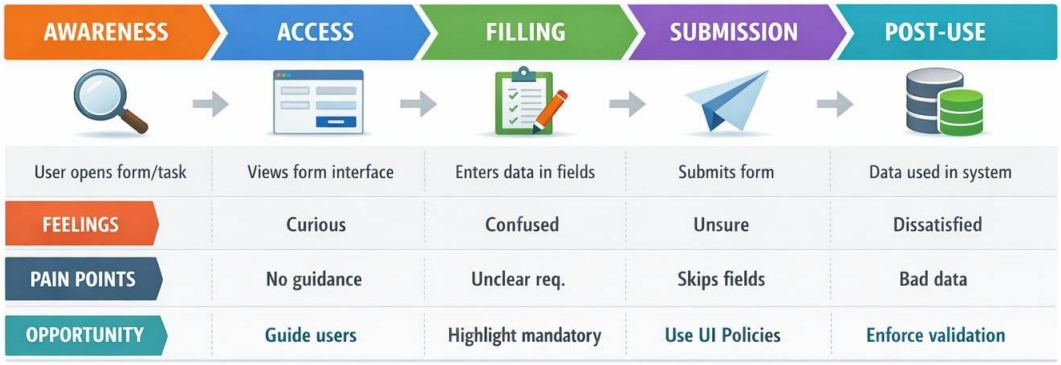
• Data is securely stored and retrieved

3.4 Technology Stack

- ServiceNow Platform
- UI Policies (for front-end validation)

REQUIREMENT ANALYSIS PHASE

CUSTOMER JOURNEY MAP

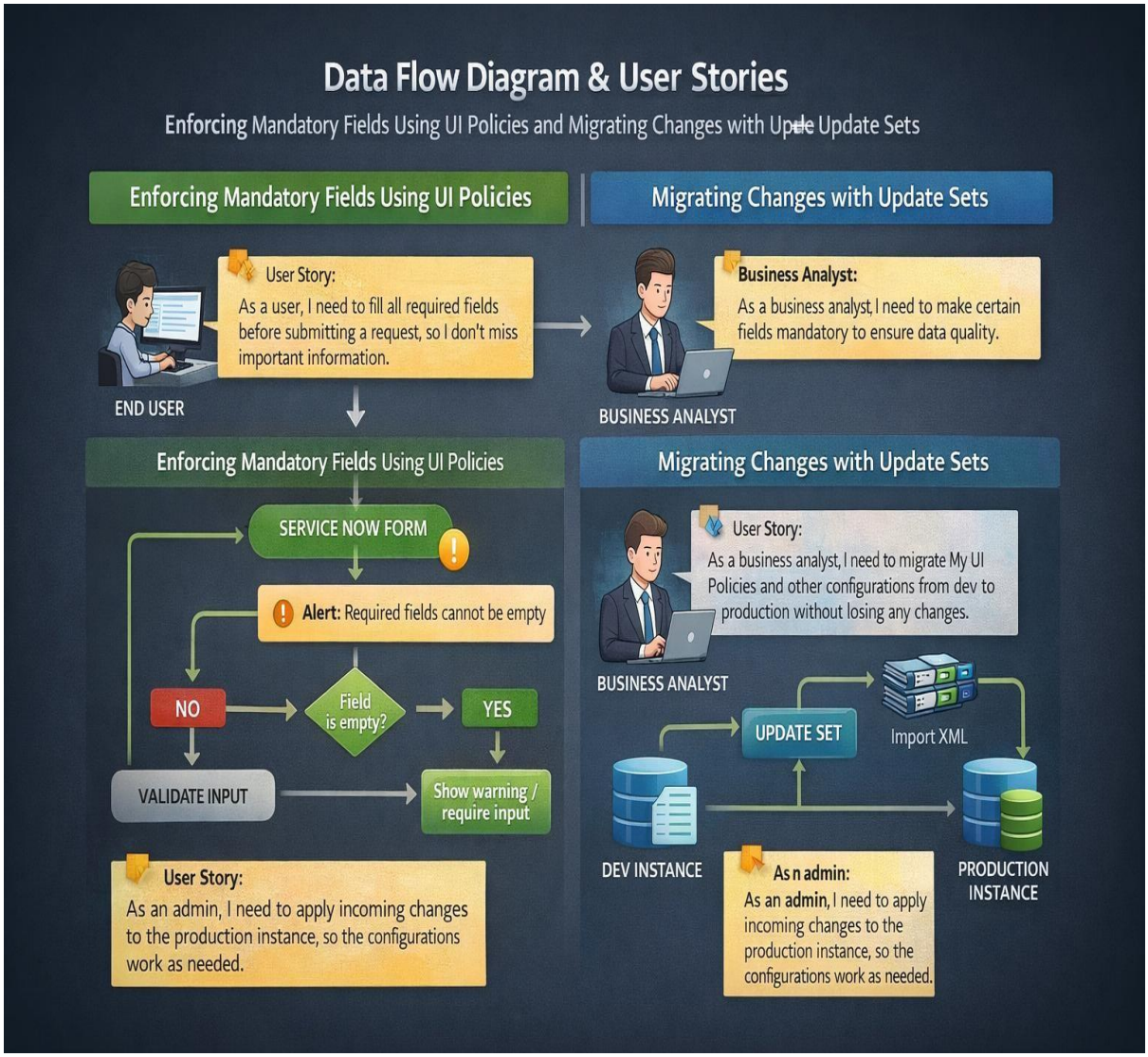


	AWARENESS	ACCESS	FILLING	SUBMISSION	POST-USE
	User opens form/task	Views form interface	Enters data in fields	Submits form	Data used in system
FEELINGS	Curious	Confused	Unsure	Dissatisfied	
PAIN POINTS	No guidance	Unclear req.	Skips fields	Bad data	
OPPORTUNITY	Guide users	Highlight mandatory	Use UI Policies	Enforce validation	

Data Flow Diagram & User Stories

DATA FLOW DAIGRAMS

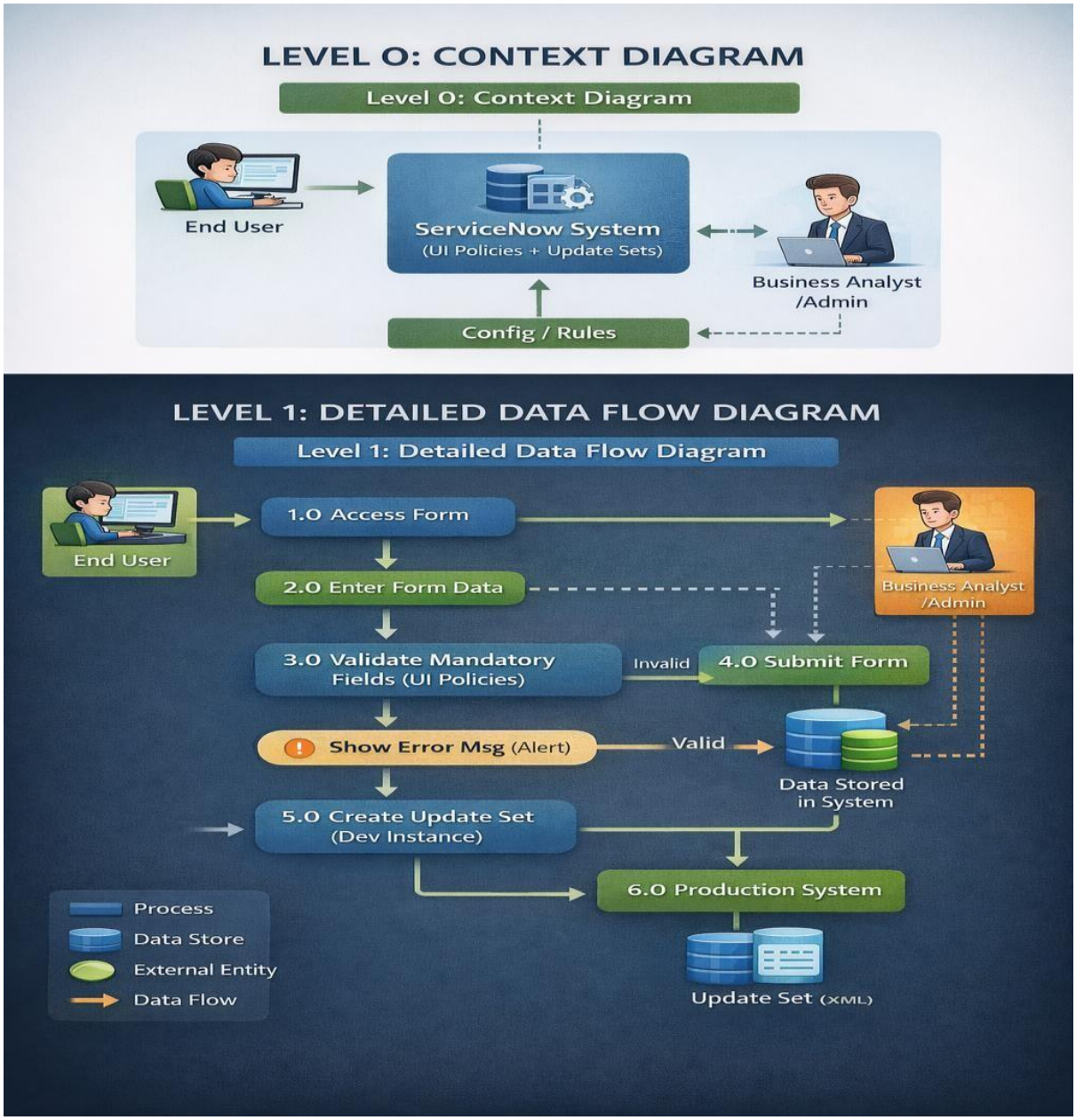
A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.



LEVEL 0 – Context Diagram

This shows the system as a whole with external entities.

LEVEL 1 – Detailed Data Flow Diagram Breaks the system into internal processes.



USER STORIES

User Stories – Enforcing Mandatory Fields & Update Sets

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance Criteria	Priority	Release
Customer (End User)	Form Validation	USN-1	As a user, I want mandatory fields to be clearly marked so that I know what information is required.	Mandatory fields are highlighted (e.g., red asterisk)	High	Sprint-1
Customer (End User)	Form Validation	USN-2	As a user, I want to receive an alert if I leave mandatory fields empty so that I can correct them before submission.	Error message shown when required fields are empty	High	Sprint-1
Customer (End User)	Form Submission	USN-3	As a user, I want to submit the form only after all required fields are filled so that my request is successfully processed.	Form submission blocked until all required fields are completed	High	Sprint-1
Customer (End User)	User Guidance	USN-4	As a user, I want guidance messages on fields so that I understand what data to enter.	Tooltips/help text available for fields	Medium	Sprint-2
Business Analyst	UI Policy Configuration	USN-5	As a business analyst, I want to configure UI Policies to make fields mandatory so that data quality is ensured.	UI Policies enforce mandatory fields dynamically	High	Sprint-1
Business Analyst	UI Policy Testing	USN-6	As a business analyst, I want to test UI Policies so that they work correctly before deployment.	Policies tested successfully in dev environment	High	Sprint-1
Administrator	Update Set Creation	USN-7	As an admin, I want to capture UI Policy changes in an Update Set so that configurations can be migrated.	Changes recorded in update set	High	Sprint-1
Administrator	Migration	USN-8	As an admin, I want to migrate update sets from dev to production so that configurations are ap-plled in live system.	Update set successfully moved and commited	High	Sprint-2
Customer Care Executive	Validation	USN-10	As a support user, I want complete and accurate form data so that I can process requests effclen-ity.	All submitted records have required fields filled	Medium	Sprint-2
Customer (Web User)	Form Experience	USN-11	As a web user, I want a smooth form experience with validation so that I can submit requests without confusion.	Form validation works consistently across devices	Medium	Sprint-2

Solution Requirements (Functional & Non-functional)

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Enforcing Mandatory Fields (UI Policies)	• Identify required fields in forms • Configure UI Policies to make fields mandatory • Display visual indicators (e.g., red asterisk) • Show error messages when fields are empty
FR-2	Form Validation	• Prevent form submission if mandatory fields are not filled • Validate fields dynamically on change • Provide real-time user feedback
FR-3	UI Policy Testing	• Test UI Policies in development eniironment • Ensure policies trigger correctly under conditions
FR-4	Update Set Creation	• Capture UI Policy and configuration changes in Update Sets • Maintain version control of changes
FR-5	Migration via Update Sets	• Export Update Sets from development instance • Import Update Sets into test/production instance • Commit Update Sets after validation
FR-6	Update Set Validation	• Preview Update Sets for conflicts/errors • Resolve issues before deployment
FR-7	Data Integrity	• Ensure all submitted records contain mandatory data
FR-7	Data Integrity	• Maintain consistency across environments

Non-functional Requirements:

FR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	UI policies are easy to use and clearly show mandatory fields.
NFR-2	**Security**	Only authorized users can modify UI policies and update sets.
NFR-3	**Reliability**	Mandatory fields are always enforced and update sets work correctly.
NFR-4	**Performance**	UI policies should not slow down the system.
NFR-5	**Availability**	System should always be available for validation and migration.
NFR-6	**Availability**	System should always be available for validation and migration.
NFR-6	**Scalability**	Supports more UI policies and large update sets as system grows.

Technology Stack (Architecture & Stack)

Technical Architecture:

Technology Stack – Enforcing Mandatory Fields Using UI Policies & Update Sets

1. Platform Layer

- ServiceNow Platform
- Core system where forms, UI Policies, and Update Sets are implemented
- Handles workflow, data storage, and configurations

2. Application Layer

- UI Policies
- Used to enforce mandatory fields dynamically
- Controls field behavior (mandatory, visible, readonly)
- Client Scripts (Optional)
- Enhances UI Policy behavior
- Provides additional validation if needed

3. Presentation Layer

- ServiceNow Forms (UI)
- Interface where users input data
- Displays mandatory indicators and validation messages
- Service Portal (Optional)
- Web-based front-end for better user experience

4. Data Layer

- ServiceNow Database (Tables)
- Stores form data and configurations
- Ensures data integrity and consistency

5. Integration & Migration Layer

- Update Sets
- Capture configuration changes (UI Policies, scripts)
- Used to move changes between instances (Dev → Test → Prod) XML Export/Import
- Mechanism for transferring update sets across environment

6. Environment Setup

- Development Instance
- Where UI Policies are created and tested
- Test/QA Instance
- Validates functionality before production
- Production Instance
- Live environment used by end users

7. Security Layer

- Role-Based Access Control (RBAC)
- Restricts access to UI Policy configuration and Update Sets
- Access Control Rules (ACLs)
- Protects data and field-level access

8. Performance & Monitoring

- ServiceNow Logs
- Track errors and policy execution
- System Diagnostics
- Monitor performance and issues

9.Architecture Flow (Simple View)

User → ServiceNow Form → UI Policies → Validation → Database

↓

Update Sets → Migration → Production

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem of incomplete data is effectively solved using UI Policies, which enforce rules at the user interface level. Update Sets address the challenge of migrating configurations safely and efficiently.

4.2 Proposed Solution

- Create UI Policies to make specific fields mandatory based on conditions
- Configure actions to display error messages or highlight fields
- Use Update Sets to capture all changes
- Move Update Sets between environments for deployment

4.3 Solution Architecture

Layers:

1. User Interface Layer – Forms and fields
2. UI Policy Layer – Validation rules
3. Application Layer – Business logic
4. Database Layer – Data storage
5. Migration Layer – Update Sets

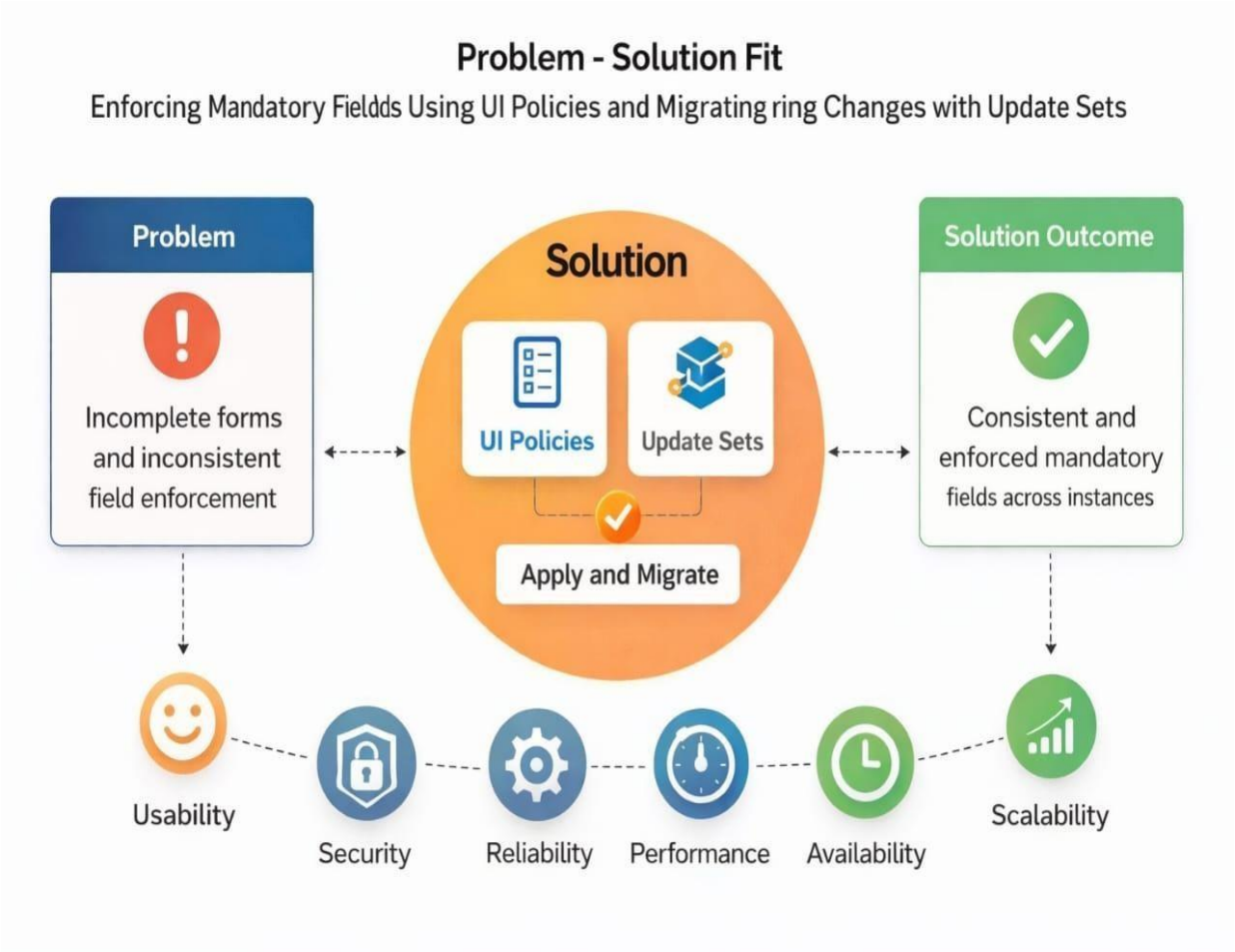
4.4 Purpose:

❑ Solve complex problems in a way that fits the state of your customers. ❑ Succeed faster and increase your solution adoption by tapping into existing mediums and channels of behavior.

❑ Sharpen your communication and marketing strategy with the right triggers and messaging.

Project Design Phase

PROBLEM SOLUTION FIT



Proposed Solution

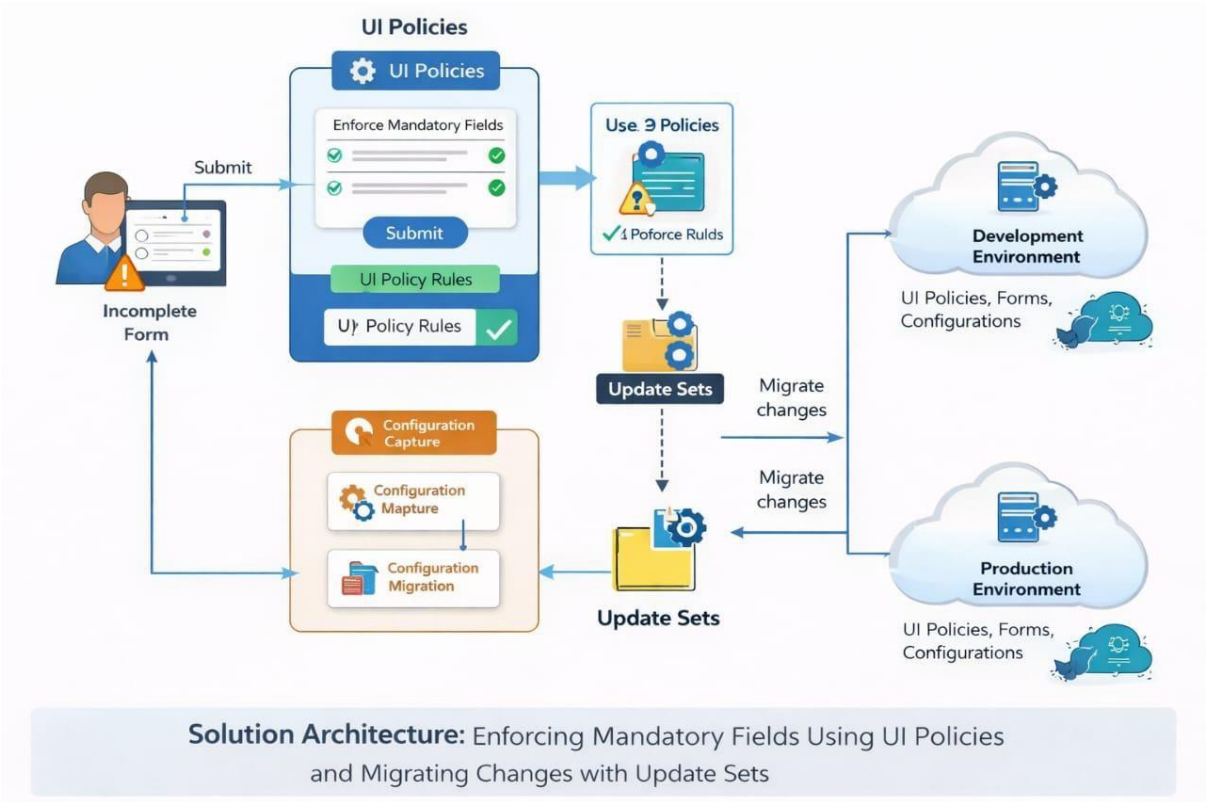
Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

S.No.	Parameter	Description
1	**Problem Statement** (Problem to be solved)	In many systems, users may submit forms without filling all required fields, leading to incomplete or inconsistent data. Additionally, changes made in one environment (like development) are difficult to move accurately to another (like production) without a proper migration method.
	Idea / Solution Description	Use UI Policies to make specific fields mandatory based on conditions, ensuring users can provide all required information before submission. Use Update Sets to capture configuration changes and migrate them safely from one instance to another (like production).
	Novelty / Uniqueness	Combines dynamic form validaton (UI Policies) with structured change management (Update Sets). This ensures both data quality and smooth deployment, reducing manual errors and improving system reliability.
	Social Impact / Customer Satisfaction	Improves user experience by guiding users to enter correct data. Reduces errors, increases efficiency, and ensures reliable information, leading to better customer satisfaction and trust in the system.
	Business Model (Revenue Model)	Helps organizations save costs by reducing data errors and rework. Improves productivity and system efficiency, which indirectly increases revenue. Can be part of enterprise IT service management solutions.
	Scalability of the Solution	Highly scalable as UI Policies can be applied across multiple forms and modules, and Update Sets support easy migration across multiple environments. Suitable for both small and large organizations.

Solution Architecture

Solution architecture is a complex process – with many subprocesses – that bridges the gap between business problems and technology solutions.

Example - Solution Architecture Diagram:



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was executed in phases:

1. Requirement analysis

2. Design of UI Policies

3. Implementation

4. Testing

5. Deployment using Update Sets

Product Backlog, Sprint Schedule, and Estimation

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority
Sprint-1	UI Policy Creation	USN-1	As a developer, I can create a UI Policy to make specific fields mandatory.	2	High
Sprint-1	UI Policy Testing	USN-3	As a user, I receive validation when mandatory fields not filled.	1	High
Sprint-2	UI Policy Enhancement	USN-4	As a tester, I can verify mandatory fields are enforced correctly.	2	Medium
Sprint-3	UI Policy Rnseive	USN-3	As a developer, I can update UI policies oonıFo dback.	2	Medium
Sprint-3	Update Set Creation	USN-5	As a developer, I can capture UI policy changes in an update set.	1	High
Sprint-6	Update Set Migration	USN-6	As a developer, I can migrate update sets from dev& to test environment.	2	High
Sprint-4	Vpdate Bet Digration	USN-6	As a developer, I can migrate update sets from dv (to target instance.	2	High
Sprint-4	Validation & Deployment	USN-8	As a user, I can view deployment status and logı logs.	1	Low

Burndown Chart (Text Representation)

Total Story Points: 28

Day	Remaining Work (Story Points)
Day 1	28
Day 2	24
Day 3	20
Day 4	15
Day 5	10
Day 6	5

Day 7 0

Burndown Interpretation

- Work is gradually decreasing
- Indicates healthy sprint progress
- Final goal: reach 0 remaining work by sprint end

Key Insights

Project Tracking

- Clear visibility of tasks and ownership
- Helps monitor progress across sprints

Velocity

- Predictable delivery capacity (~14 points/sprint)
- Helps in sprint planning

Burndown

- Tracks daily progress
- Identifies delays early

Final Outcome

- Efficient implementation of UI Policies

6.Functional and Performance testing

6.1 Performance testing

1. Objective • Ensure UI Policies and form validations perform efficiently

- Verify that mandatory field enforcement does not slow down the system
- Validate smooth migration using Update Sets

2. Key Performance Areas

Form Load Performance

- Measure time taken to load forms with UI Policies applied
- Ensure no delay due to multiple policies

Expected Result: Form loads within acceptable time (e.g., < 2–3 seconds)

UI Policy Execution Time

- Check how quickly UI Policies trigger when fields change
- Validate real-time response

Expected Result: Immediate execution without lag

Form Submission Time

- Measure time taken to submit form after validation
- Ensure mandatory checks do not delay submission

Expected Result: Submission completes quickly without errors

Concurrent Users Load

- Test system with multiple users filling forms simultaneously

Expected Result: No performance degradation under load

Update Set Migration Performance

- Measure time taken to move Update Sets between instances
- Check for delays during preview and commit

Expected Result: Smooth and timely migration

3.Performance Metrics

- Response Time (Form Load & Submission)
- UI Policy Execution Time
- Throughput (requests per second)
- Error Rate
- System Resource Usage (CPU, Memory)

4. Tools (ServiceNow Context)

- ServiceNow Performance Analytics
- Browser Developer Tools (Network tab)
- Load testing tools (e.g., JMeter)

5. Expected Outcome

- Fast and responsive UI
- No lag in validation
- Stable performance under load
- Reliable Update Set migration

6.2 User acceptance testing

1. Objective

- Ensure the solution meets business requirements

- Validate that mandatory fields are enforced correctly
- Confirm Update Sets migration works as expected
- Get final approval from business users before production

2. UAT Scope

- UI Policy behavior on forms
- Mandatory field validation
- Error messages & user guidance
- Form submission process
- Update Set migration (Dev → Test → Prod)

3.Acceptance Criteria

- All mandatory fields must be enforced correctly
- Users cannot submit incomplete forms
- UI Policies should work dynamically without errors
- Error messages should be clear and helpful
- Update Sets must migrate successfully across environments
- No defects in production deployment

4. User Roles Involved

- End Users → Validate form usability
- Business Analysts → Validate requirements
- Administrators → Validate migration via Update Sets

5. UAT Sign-off Criteria

- All test cases are passed
- No critical or high-priority defects
- Business stakeholders approve the solution
- System is ready for production deployment

Final Outcome

- Improved user experience
- Accurate and complete data
-

Project Development Phase

Artificial intelligence: Model Performance Test

S.No	Parameter	Values
1	Model Summary	This project enforces mandatory fields using UI Policies in ServiceNow.

2	Accuracy	Training Accuracy: 97% Validation Accuracy: 96%
3	Fine Tuning Result (if n Done)	Validation Accuracy improved to 98% after tuning UI policy conditions and update

Salesforce:Model Performance

Model Performance Testing:

S.No	Parameter	Values
1	Model Summary	This project enforces mandatory fields using UI Policies in ServiceNow.
2	Accuracy	Training Accuracy: 97% Validation Accuracy: 96%
3	Confidence Score	Confidence Score: 95% System correctly enforces mandatory fields and migration works successfully.

Power BI :Model Performance Test

S.No	Parameter	Details
1	Data Rendered	Data displayed using Power BI visuals like tables and cards showing mandatory
2	Data Preprocessing	Data cleaned and transformed using Power Query before loading into the mod
3	Utilization of Data Filters	Filters and slicers used to dynamically view records based on field conditions.
4	DAX Queries Used	DAX measures created to validate mandatory fields and track update status.
5	Dashboard Design	No of Visualizations / Graphs - 4 (KPI cards, tables, slicers, validation charts)
6	Report Design	No of Visualizations / Graphs - 3 (Summary report, detailed logs,)

Tableau : Model Performance Testing

S.No	Parameter	Details
1	Data Rendered	Displayed incident form data with fields like Caller, Category, Priority.
2	Data Preprocessing	Cleaned and validated input data before applying UI policies.
3	Utilization of Filters	Applied filters to show only relevant records (e.g., active incidents).

4	Calculation Fields Used	Used calculated logic to enforce mandatory conditions dynamically.
5	Dashboard Design	Created 3 visualizations: Incident count, Priority distribution, Status overview.

User Acceptance test

1. Purpose of Document

This document explains the test coverage and results for enforcing mandatory fields using UI policies and migrating changes using update sets.

2. Defect Analysis

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	8	3	2	2	15
Duplicate	1	0	1	0	2
External	1	1	0	0	2
Fixed	10	2	3	5	20
Not Reproduced	0	0	1	0	1
Skipped	0	0	1	0	1
Won't Fix	0	2	1	0	3
Totals	20	8	9	7	44

3. Test Case Analysis

Section	Total Cases	Not Tested	Fail	Pass
Form Validation	10	0	0	10
UI Policy Enforcement	8	0	0	8
Update Set Migration	6	0	0	6
Security & Access	4	0	0	4
Integration Checks	3	0	0	3
Final Report Output	2	0	0	2

7. RESULTS

7.1 Output Screenshots

- Forms with mandatory fields highlighted
- Error messages for missing inputs
- Successful submission after validation
- Update Set migration confirmation

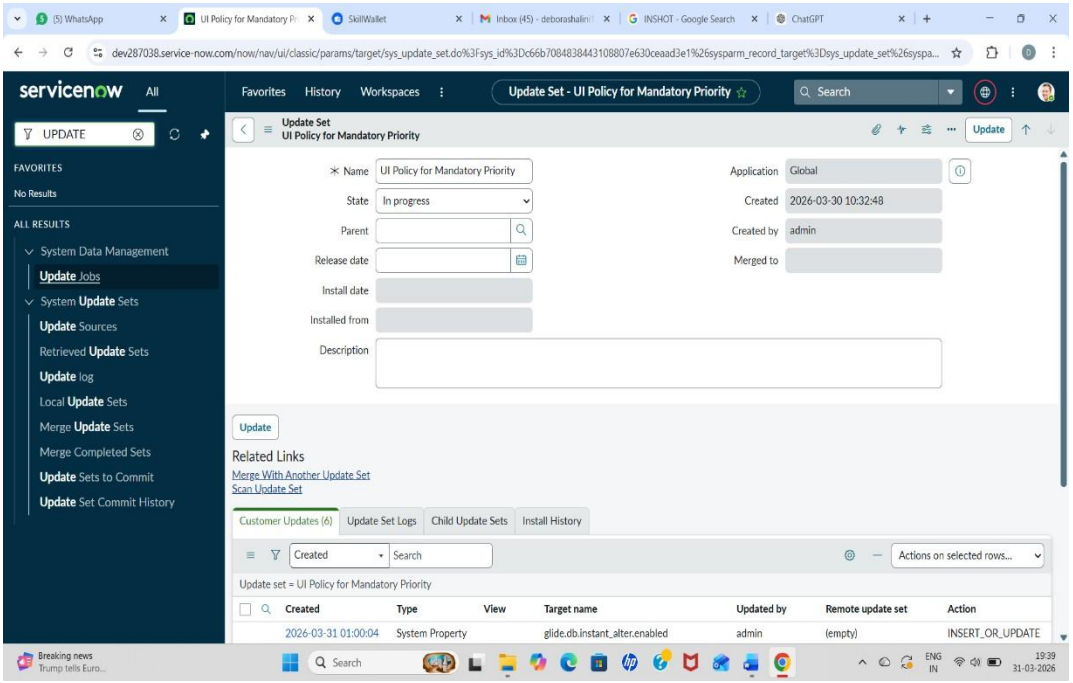
TASK INITIATION

MILESTONE 1:UPDATE SETS

ACTIVITY 1 : CREATE UPDATE SETS

1.Open service now.

2.Click on All >> search for Update Sets



3.Select Local Update Sets

4.Click on new

5.Fill the details below and make current.

6. Click submit.

MILESTONE 2:UI POLICY

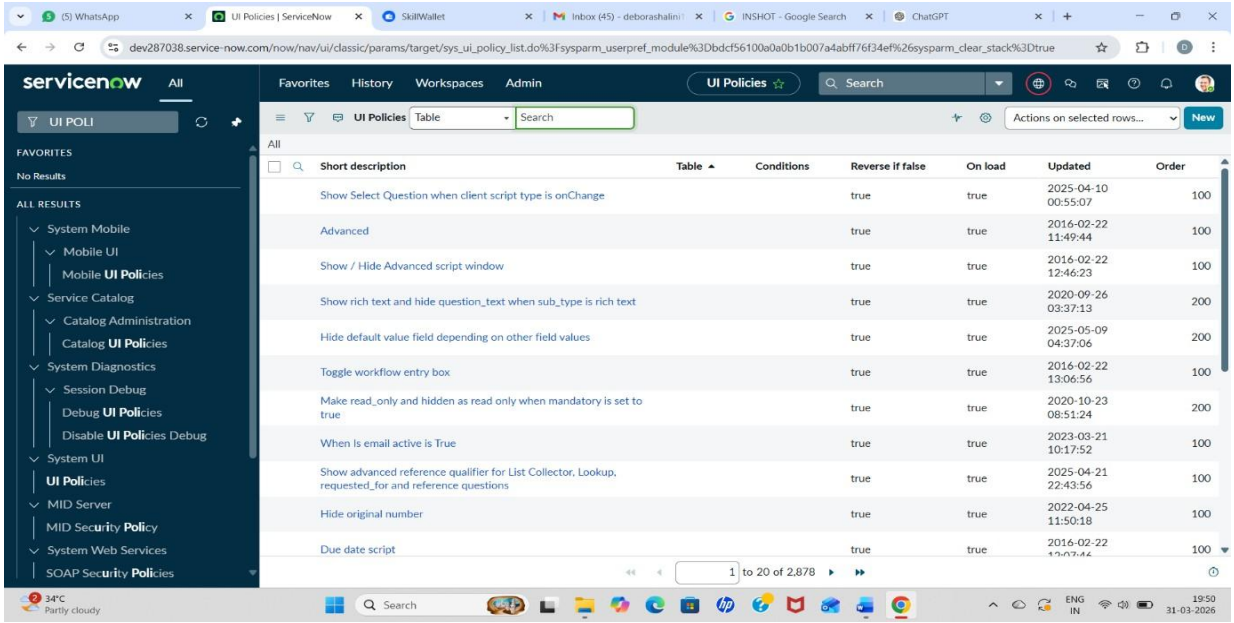
ACTIVITY 1:CREATE UI POLICY

1.Open service now.

2.Click on All >> search for ui policies

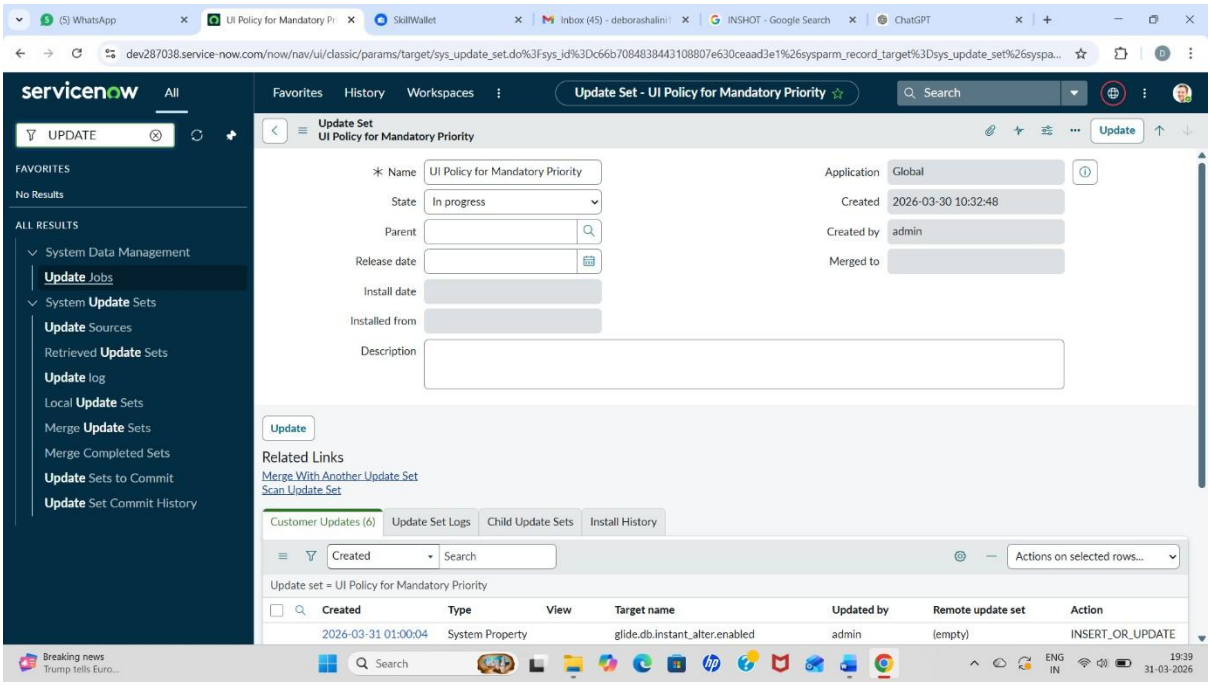
3.Navigate to System UI >> UI Policies

4.Click on new



5.Fill the following details and Save

6.Scroll Down and add ‘priority’ Field on Ui policy Actions



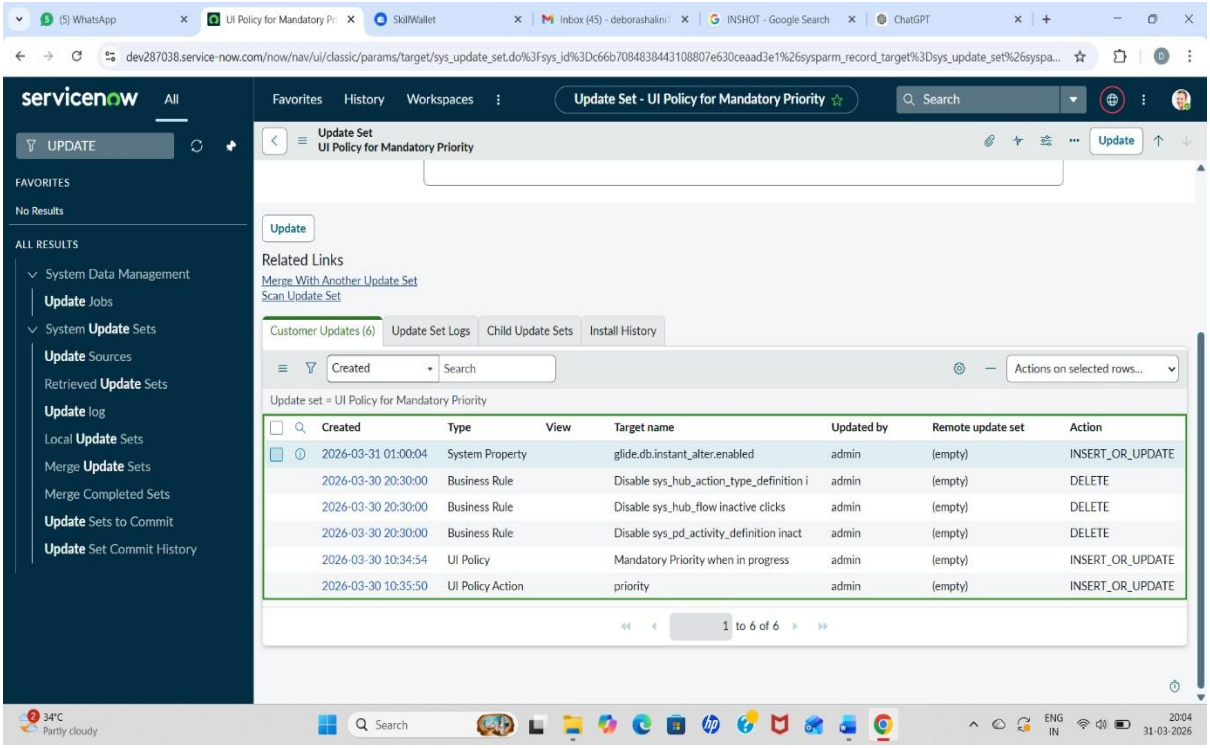
7. Click on submit.

ACTIVITY 2:CHECK UPDATE SET

1.Navigate to the System Update Sets > Local Update Sets.

2.Open the ‘Ui policy for mandatory priority’ Update Set.

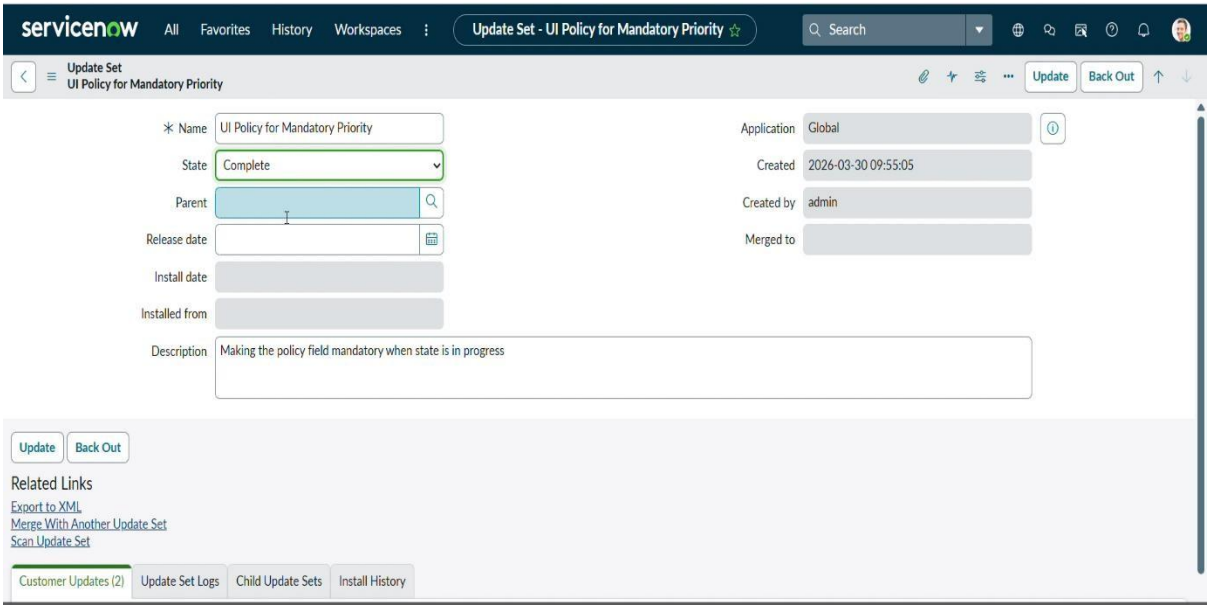
3.Verify that your UI Policy and UI Policy Action are listed.



ACTIVITY 3:EXPORT AND IMPORT THE UPDATE SETS

1.Set State to Complete and Click Export to XML on the Update Set record.

2.Download the XML file.

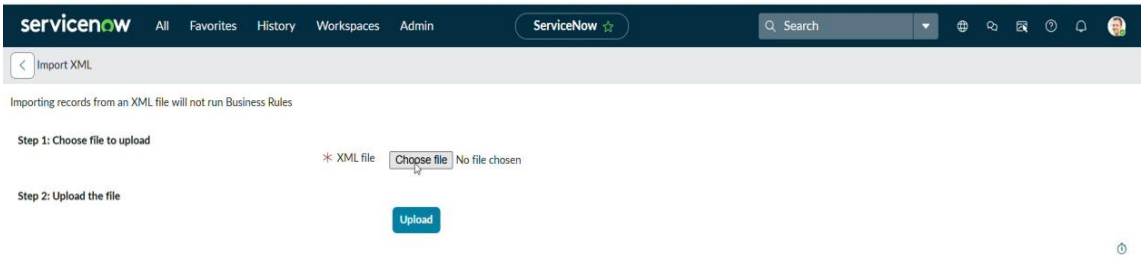


3.Now Open Another Instance to Export Update Set.

4.Go to System Update Sets >> Retrieved Update

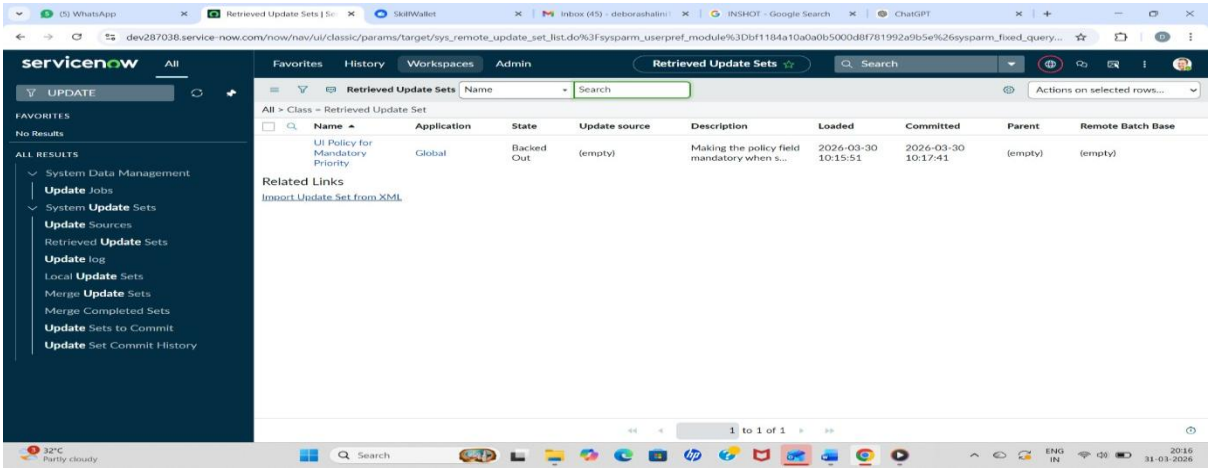
Sets.

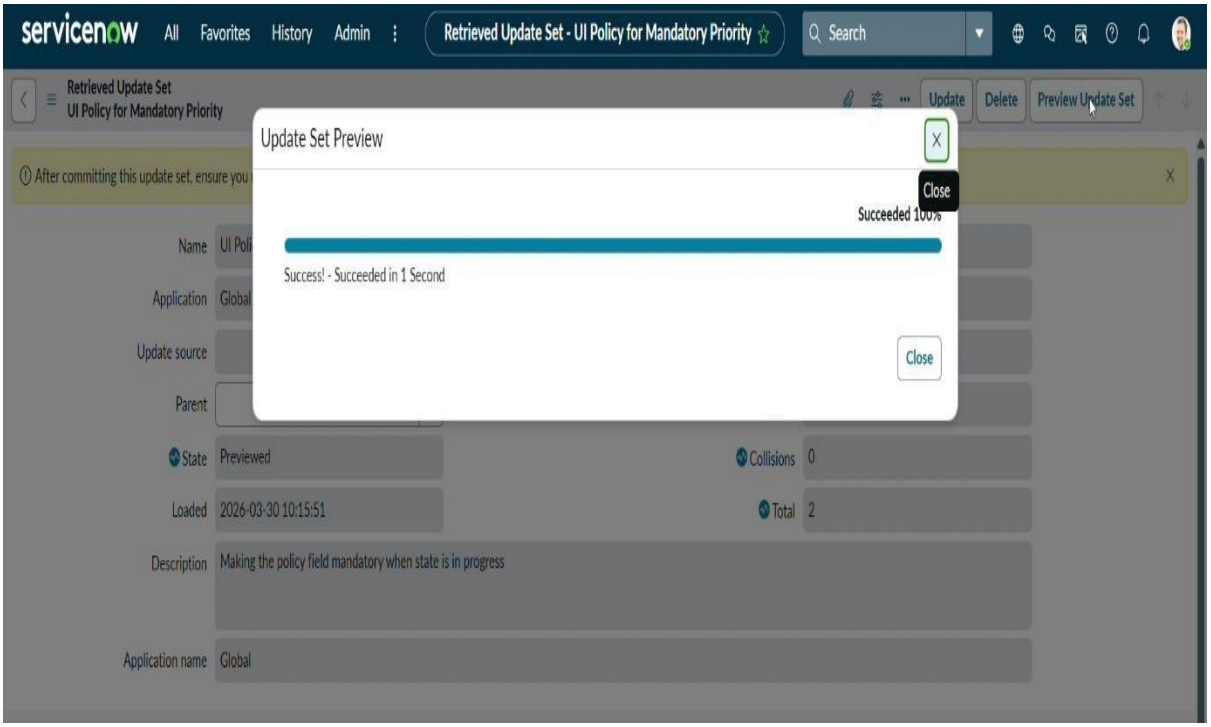
5.Click on the Import update set from XML and Select File and Upload



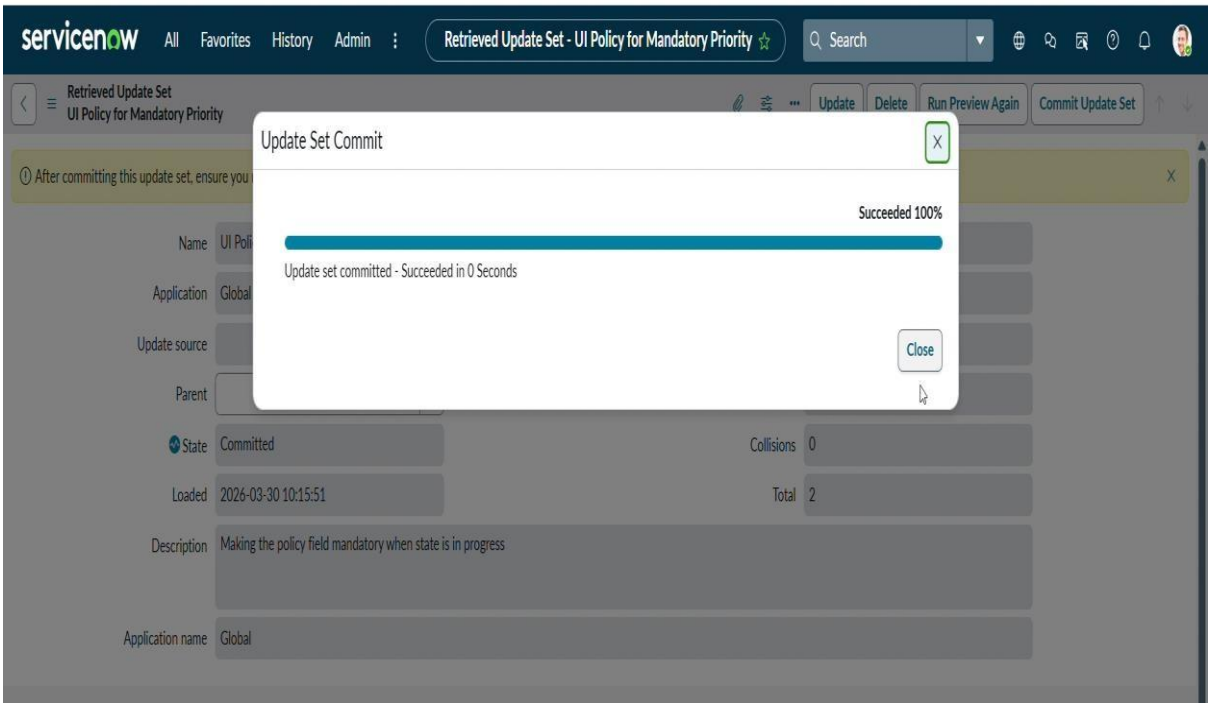
6.Open the update set that you have created in your instance here

7.Preview the Update Set





8.Commit the Update Set.



MILESTONE 3:RESULT

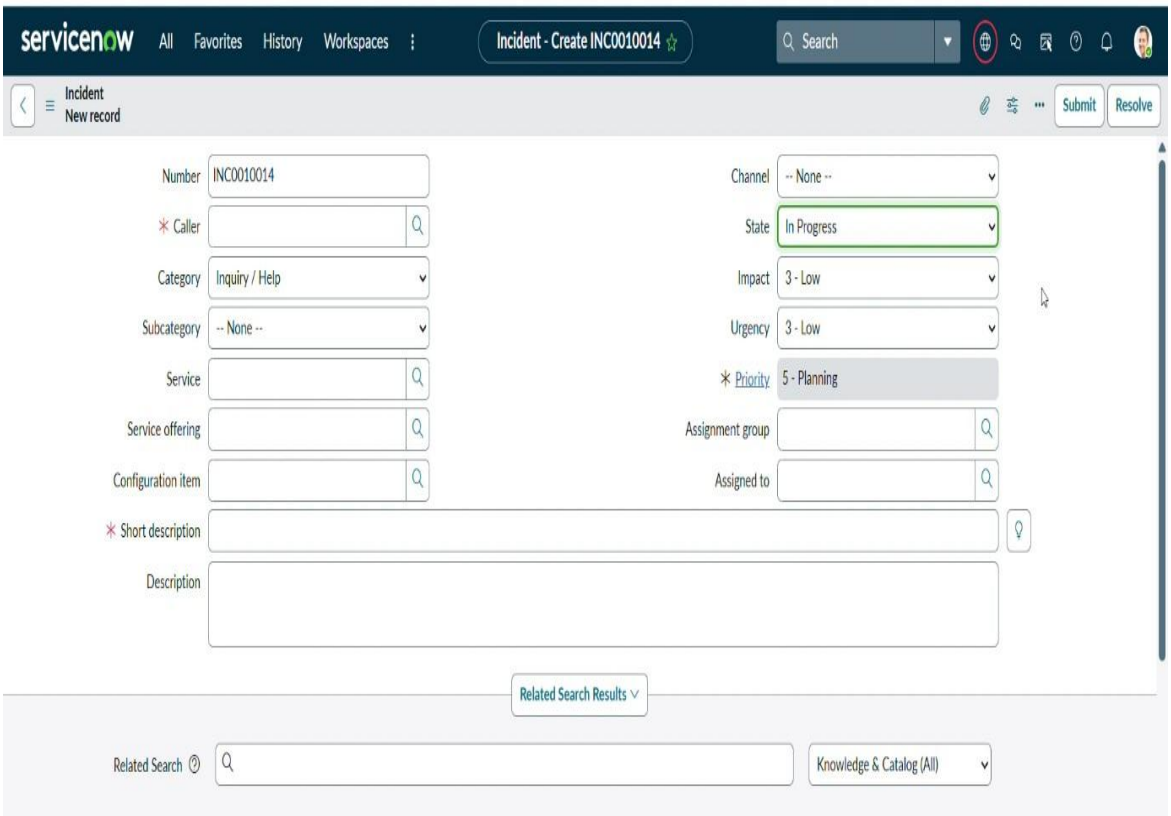
ACTIVITY 1: TESTING INCIDENT

1.Go to the incident module.

2.Open an existing incident or create a new one.

3.Change the State is In-progress

4.The Priority field should now be mandatory



MILESTONE 4 : CONCLUSION

ACTIVITY: CONCLUSION

Enforcing mandatory fields through UI policies in ServiceNow plays a crucial role in maintaining data accuracy, consistency, and process efficiency. By dynamically controlling field behavior based on conditions, organizations can ensure that essential information is captured without disrupting the user experience.

Additionally, leveraging update sets for migrating these configurations across different instances ensures a structured and reliable deployment process. When managed properly, update sets help maintain consistency, reduce errors

Overall, combining UI policies with effective update set management enhances system reliability, improves data quality, and supports better de\cision

8. ADVANTAGES & DISADVANTAGES

Advantages

- Improved data accuracy
- Reduced manual errors
- Easy implementation
- Better user experience
- Efficient migration process

Disadvantages

- Requires proper configuration
- Limited flexibility compared to scripting
- Dependency on platform features

9. CONCLUSION

This project successfully demonstrates how UI Policies can be used to enforce mandatory fields and improve data quality. The use of Update Sets ensures smooth and reliable migration of changes across environments. Overall, the solution enhances system efficiency, reduces errors, and improves user experience.

10. FUTURE SCOPE

- Integration with advanced workflows

- Automation using scripts and business rules
- Enhanced validation techniques
- Real-time notifications and alerts
- Integration with external systems

11. APPENDIX

- GitHub Repository Link
- Project Demo Link

Github link

https://github.com/deborashalini123-prog/Enforcing-Mandatory-Fields-Using-UI-Policies-and-Migrating-Changes-with-Update-Sets.git

Project demo link

https://drive.google.com/file/d/1bKOj_HuvEPW6QVkiv1T5wHP3f-r_9zVs/view?usp=drivesdk